

计算器功能实现

接下来我们补充实现 `setOperator`（选择运算符）、`calculate`（计算结果）以及其他功能方法的完整逻辑：

② `setOperator`：处理运算符选择

```
private void setOperator(String op) {  
    // 如果还没有输入第一个数字，不执行任何操作  
    if (firstNum.isEmpty()) {  
        return;  
    }  
  
    // 如果已经输入了第二个数字，先计算之前的结果，再用结果作为新的第一个操作数  
    if (!secondNum.isEmpty()) {  
        calculate();  
        // 计算后将结果作为新的第一个操作数  
        firstNum = resultTv.getText().toString();  
        secondNum = ""; // 清空第二个操作数  
    }  
  
    // 记录选择的运算符，并标记已点击运算符  
    operator = op;  
    isOperatorClicked = true;  
}
```

③ `calculate`：执行计算逻辑

```
private void calculate() {  
    // 检查是否有完整的计算条件（必须有两个操作数和运算符）  
    if (firstNum.isEmpty() || operator.isEmpty() || secondNum.isEmpty()) {  
        return;  
    }  
  
    double result = 0;  
    double num1 = Double.parseDouble(firstNum);  
    double num2 = Double.parseDouble(secondNum);
```

```
// 根据运算符执行对应计算
switch (operator) {
    case "+":
        result = num1 + num2;
        break;
    case "-":
        result = num1 - num2;
        break;
    case "×":
        result = num1 * num2;
        break;
    case "÷":
        // 处理除数为0的情况
        if (num2 == 0) {
            resultTv.setText("错误");
            clear(); // 清空所有数据
            return;
        }
        result = num1 / num2;
        break;
}

// 处理结果显示（如果是整数则去掉小数点，否则保留原精度）
String resultStr;
if (result == (long) result) {
    resultStr = String.valueOf((long) result);
} else {
    resultStr = String.valueOf(result);
}

// 更新显示并重置状态
resultTv.setText(resultStr);
firstNum = resultStr; // 计算结果作为下一次计算的第一个操作数
operator = ""; // 清空运算符
secondNum = ""; // 清空第二个操作数
isOperatorClicked = false;
}
```

④ 其他功能方法实现

```
// 清除所有数据（C键功能）
private void clear() {
    firstNum = "";
    operator = "";
    secondNum = "";
    isOperatorClicked = false;
    resultTv.setText("0"); // 初始显示0
}

// 切换正负号（±键功能）
private void changeSign() {
    if (isOperatorClicked) {
        // 处理第二个操作数的正负号
        if (!secondNum.isEmpty()) {
            if (secondNum.startsWith("-")) {
                secondNum = secondNum.substring(1);
            } else {
                secondNum = "-" + secondNum;
            }
            resultTv.setText(secondNum);
        }
    } else {
        // 处理第一个操作数的正负号
        if (!firstNum.isEmpty()) {
            if (firstNum.startsWith("-")) {
                firstNum = firstNum.substring(1);
            } else {
                firstNum = "-" + firstNum;
            }
            resultTv.setText(firstNum);
        }
    }
}

// 百分比计算（%键功能）
private void setPercent() {
    if (isOperatorClicked) {
        // 第二个操作数转换为百分比（除以100）
        if (!secondNum.isEmpty()) {
```

```
        double num = Double.parseDouble(secondNum) / 100;
        secondNum = String.valueOf(num);
        resultTv.setText(secondNum);
    }
} else {
    // 第一个操作数转换为百分比（除以100）
    if (!firstNum.isEmpty()) {
        double num = Double.parseDouble(firstNum) / 100;
        firstNum = String.valueOf(num);
        resultTv.setText(firstNum);
    }
}
}
```

逻辑说明

1. setOperator:

- 确保先输入第一个数字后才能选择运算符
- 如果已经输入了第二个数字，会先计算当前结果，再用结果作为新的第一个操作数（支持连续计算）

2. calculate:

- 严格检查计算条件（两个操作数 + 运算符）
- 处理除法时除数为 0 的错误情况
- 优化结果显示（整数去掉小数点，避免显示 .0）

3. 其他功能:

- 清除键（C）重置所有数据并显示 0
- 正负号键（±）切换当前输入数字的正负
- 百分比键（%）将当前数字除以 100（如 "25" 变为 "0.25"）

这些方法配合之前的数字输入逻辑，就能实现一个完整的计算器功能了。

（注：文档部分内容可能由 AI 生成）